University of Glasgow

Messaging and Protocols

Internet Technology

ITECH

By the end of this week…

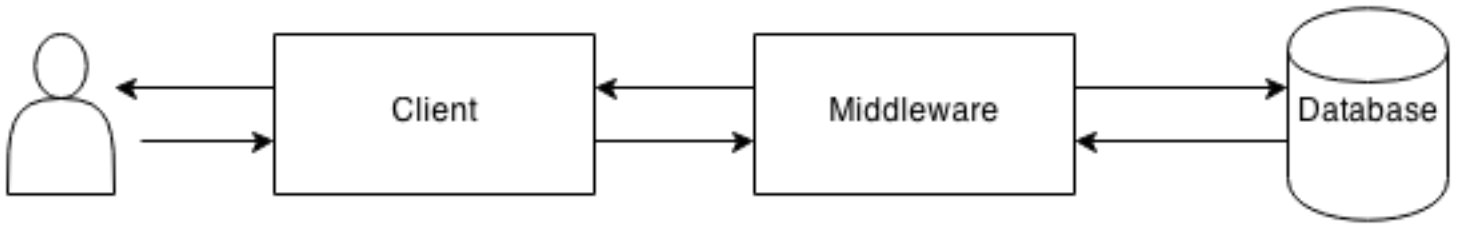
By the end of Week 4 you should be able to:

- Explain the fundamentals of network communication
- Understand HTTP and its key characteristics
- Differentiate between HTTP GET and POST
- Understand Inter-Process Communication and specifically RESTful APIs
- Use forms and URLs in Django (own-study, TwD Chapters 7 & 8)

Resources

- This lecture uses some material from the book "Computer Networking: A Top-Down Approach", 8th edition, by Jim Kurose & Keith Ross (Pearson, 2020) and book powerpoint slides: https://gaia.cs.umass.edu/kurose_ross/ppt.php
- If you want to dive deeper into networking and the Application Layer, see Chapters 1 and 2 from the book above

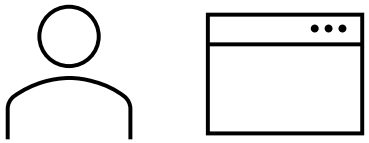
System Architecture



- We have not yet specified:
  - how messages are going to be sent, and
  - in what format these messages are going to be.
- Different interactions will require using different message formats and protocols.

What happens when you click a URL?

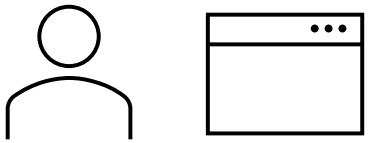
Client



Request

The user types a URL: https://www.gla.ac.uk/study

Client



Response

The webpage is displayed in the browser

A Web Application is a Network Application

Clients:

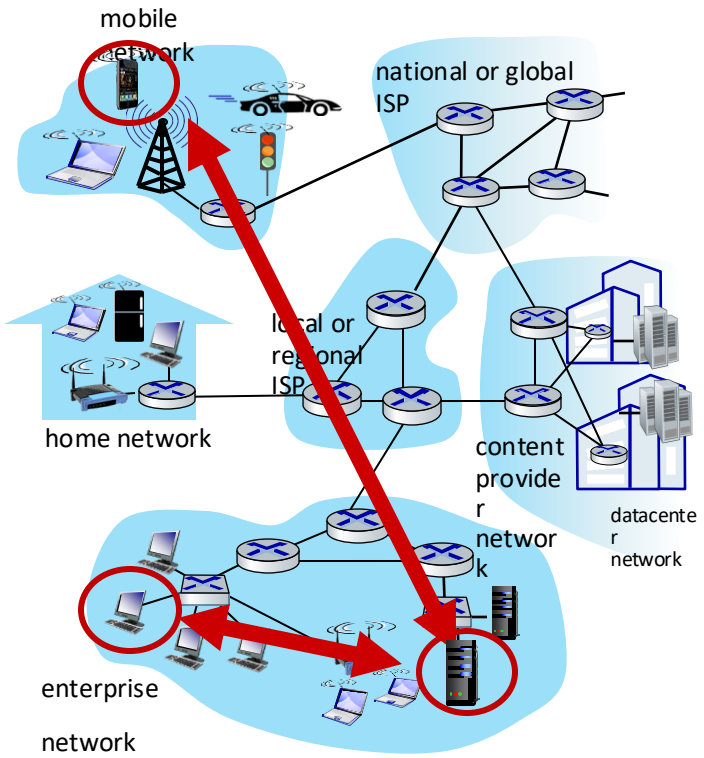
- Contact, communicate with server
- Intermittently connected
- Do NOT communicate directly with each other

Servers / Middleware / Database:

- Always-on host
- Often in datacenters for scaling

Clients **communicate** with the middleware over the **network**

- Message exchange!
- Messages follow rules!



The TCP/IP model: 4 layers for network communication

TCP/IP Model

Application

Format data so it can be processed by the application (client)

Protocols:

- **HTTP/HTTPS:** Webpages
- **SMTP, IMAP, POP3:** Emails
- **FTP:** Files
- **DNS:** URL to IP translation

Transport

Split data to packets/Check packets

Protocols:

- **TCP:** Reliable packet transmission
- **UDP:** Unreliable packet transmission

Internet

Assign IP address and route packets

Protocols:

- **IP:** Deliver packets according to IP address

Link/Physical

Prepare packets for transmission over the physical network

We will discuss:

- Application Layer
  - HTTP
  - DNS

- Some elements of the transport and internet layer
  - TCP/IP

Protocols

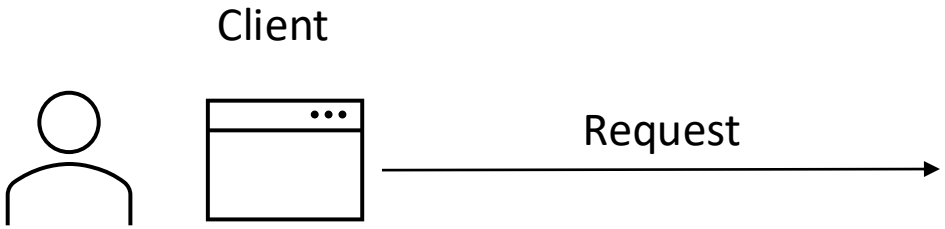
Protocols are set of conventions that determine how data is formatted, transmitted, and received over the network

- **Type of messages**, e.g. request/response
- **Message syntax**, e.g. fields in messages
- **Message semantics**, e.g. what fields mean
- **Rules** for when and how processes send & respond to messages

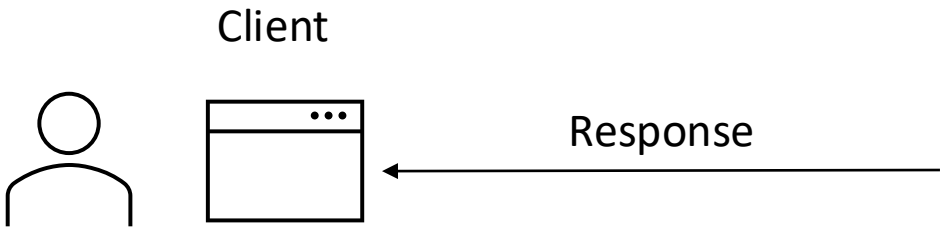
Request – Response Pattern

- A **Request-Response** pattern is a way to exchange messages.
  - A requester sends a request message (for a resource or an action)
  - The receiver of that message provides a message in response (with the resource, action, or just an acknowledgement of reception)
- Ideally, this should be performed in a synchronous fashion, however, it is asynchronous

What happens when you click a URL?



The user types a URL: https://www.gla.ac.uk/study



The webpage is displayed in the browser

- What type of request/response is this?
- What messages are exchanged before the webpage is displayed?
- What is the flow of messages?
- What protocols are involved?

Components of a URL

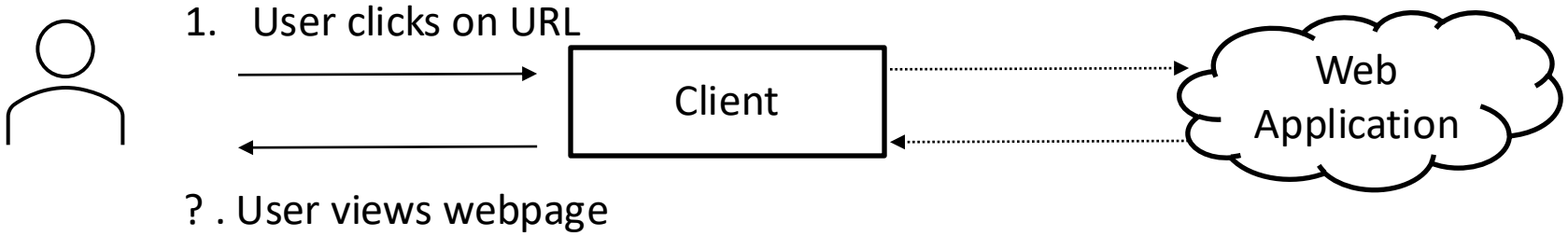
URL stands for Uniform Resource **Locator**

- An example: https://www.gla.ac.uk/study

Protocol	Domain	Path to resource
https://	**gla.ac.uk**	**/study**

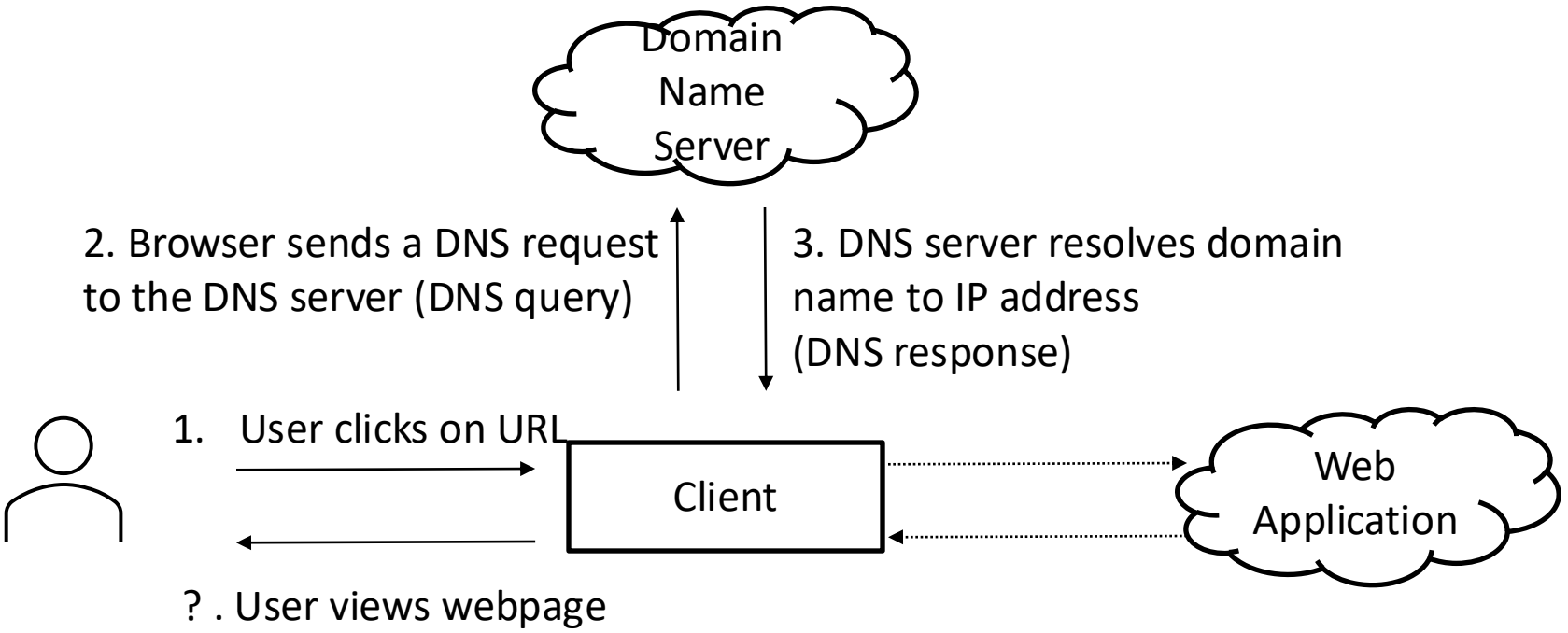
- **HTTPS:** The secure version of the HTTP protocol
- **Domain:** The location of the web application (where the server, middleware, application logic live)
- **Path:** The location of the resource

Clicking on a URL – HTTP messages



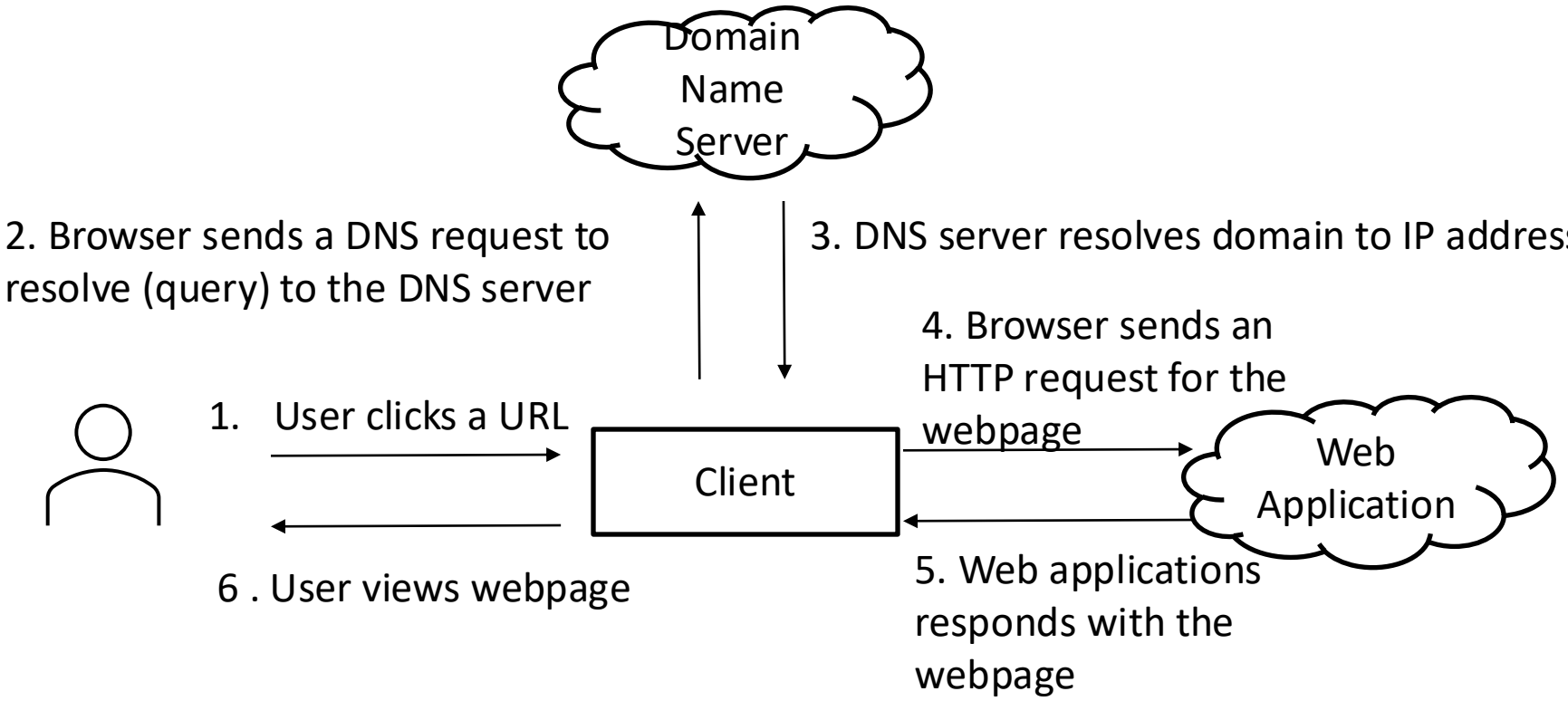
- What is the **location** of the Web Application?
  - The **Domain name** (e.g. gla.ac.uk), but the internet's languages is **IP addresses**
  - **DNS: Domain Name System** – A DNS server will translate the domain name to an IP address

Clicking on a URL – DNS messages



- The DNS server will respond with the IP address
  e.g. gla.ac.uk -> 130.209.16.90
- The **client** can now **send the HTTP request** to the web application

Clicking on a URL – Flow of messages



- A few more things are happening under the hood...
- A connection needs to be established between the Client and the Web Application/Server -> TCP

Requests and Responses (1)

The following happens when the **user agent** (i.e. the web browser/client) is asked to **send** a **message** (i.e. when the user clicks a link):

- First, the URL is turned into an **IP address**
  - **Request:** ask Domain Name System (DNS) for IP
  - **Response:** returns the IP for the URL
    - E.g. www.gla.ac.uk maps to 130.209.16.90

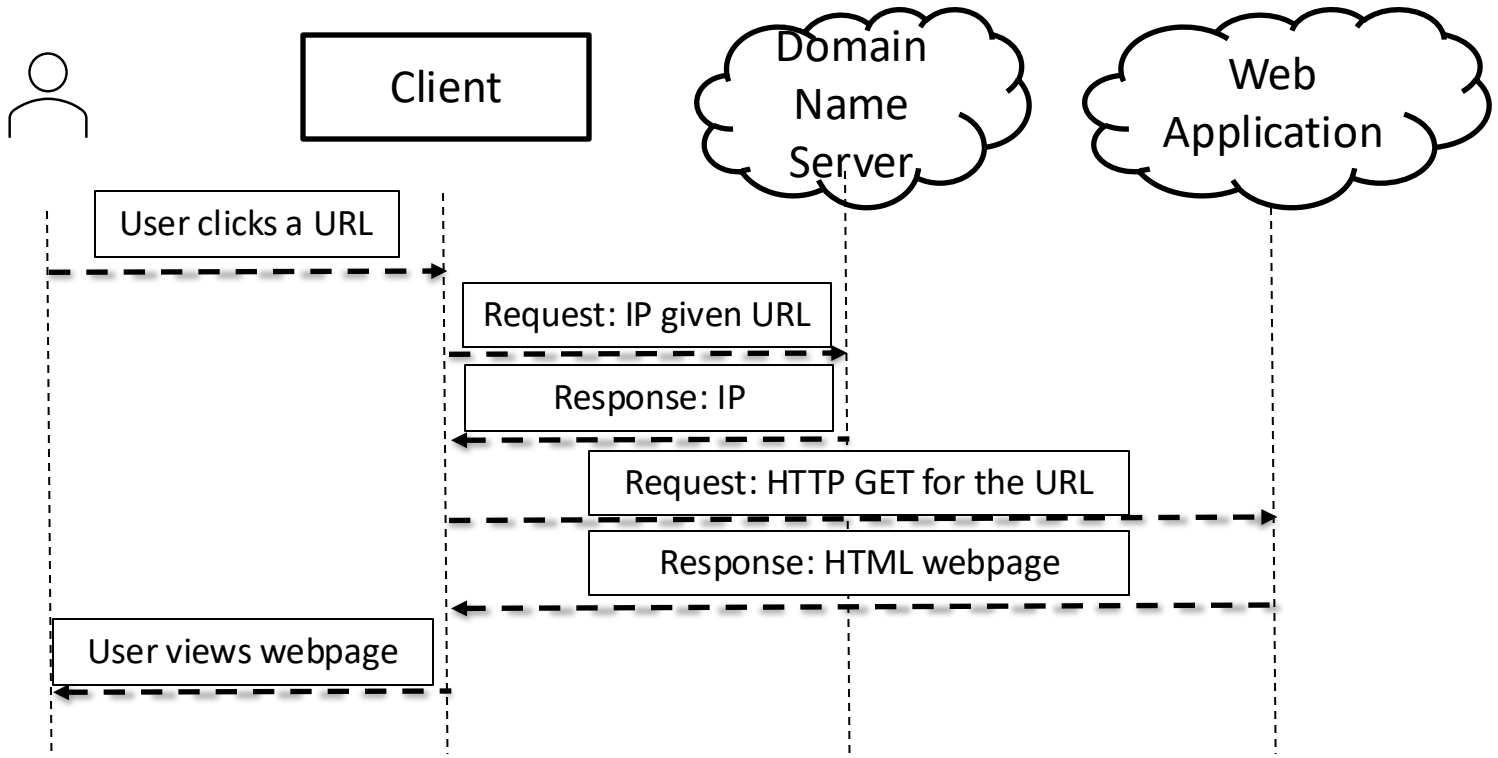
Request and Responses (2)

- Second, a **TCP connection** is opened on a particular port on the node at that IP address
  - port 80 for HTTP (standard)
  - port 443 for HTTPS (encrypted through SSL)

- Next, a request is made using a specific URL Scheme (i.e. HTTP) and sent using that TCP connection.
  - **Request:** Get the home page of the specified URL.
  - **Response:** Returns the HTML for the home page

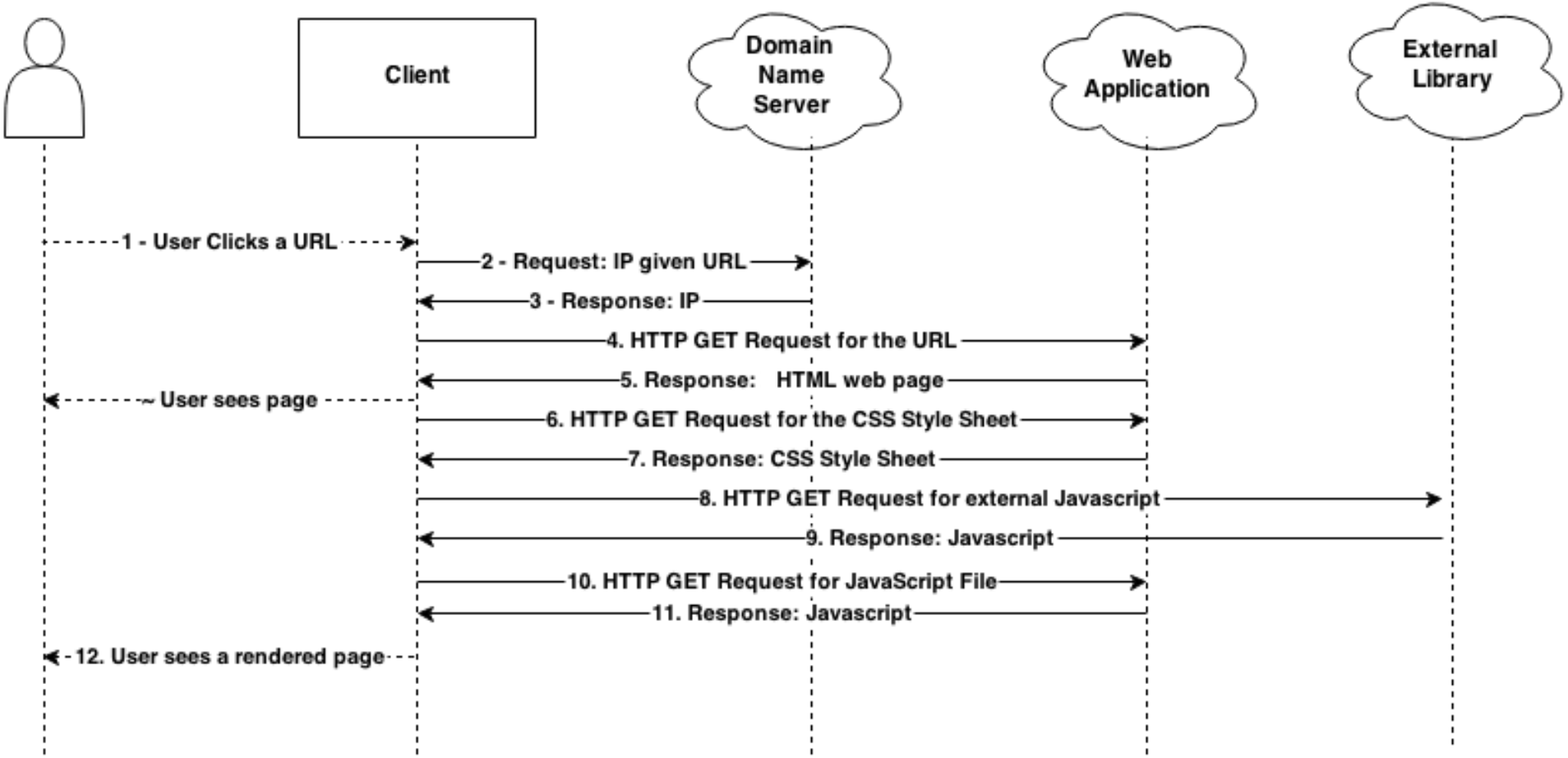
For a full ports list: https://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers

Sequence Diagrams



- We can show the flow of messages on the System Architecture Diagram, but many details are hidden
- Sequence Diagrams provide a better way to show the flow of messages
  - Label all messages

Sequence Diagrams – an example



- Further requests for Style Sheets and Javascript

Application-layer Protocols

- **Requests can be made using various protocols**
  - **http**: This is the most common protocol and indicates a file that a web browser can format and display - an HTML file, image file, sound file, etc.
  - **https**: Automatically performs Secure Socket Layer negotiation and always sends data in encrypted form
  - **file**: This indicates a file which is not in a recognised web format and will be displayed as text
  - Other protocols: ftp, mailto, news, telnet, ...

At the Application Layer

We shall be focusing on a protocol for communication and the payload formats:

1. **Hypertext Transfer Protocol (HTTP)**
   - The low level application protocol used to send messages
   - This is a specific URL Scheme
   - Follows a Request-Response pattern
   - Provides a number of ways to make a request (GET, POST)
2. **User Agent Specific Protocols (UASP)**
   - These package/wrap the information that will be sent when providing a response.
   - Such as XML, XHTML, JSON

HTTP

HTTP or **Hyper Text Transfer Protocol** is used to deliver virtually all files and other data using **8 bit characters**

- Usually, HTTP takes place through TCP/IP sockets

HTTP is used to transmit **resources**, not just files.

- A resource is some chunk of information that can be identified by a URL

- HTTP functions as a **request-response pattern** in the **client-server** computing model

HTTP Request and Response Messages

- Under HTTP, communication is as follows:
  - An HTTP client opens a connection and sends a **request message** to an HTTP server
    - Request messages are typically **GET** or **POST** methods
  - The server then returns a **response message**, usually containing the resource that was requested.
    - Respones are typically in XHTML, XML, but also in other formats like JSON
  - After delivering the response, the server **closes the connection** making HTTP a stateless protocol
    - i.e. not maintaining any connection information between transactions

HTTP GET (1)

- **HTTP GET**
  - GET retrieves data from the server
  - Should be used for operations that don't change server data

- How HTTP GET works
  - GET appends the data to the URL as key-value pairs
  - Syntax: URL ? key1 = value1 & key2 = value2
  - Example: https://www.google.com/search?q=itech
  - URL encoding: special characters within value are replaced
    - e.g. space is replaced by %20

HTTP GET (2)

- **User benefits:** The data is part of the URL – the user can see them, copy them, bookmark them, resubmit the page easily
- **When to use GET?** GET should be used for pages which **don't change anything on the server**
  - Example: Information requests
    - GET /user/profile?name=JaneDoe&address=123+Main+St HTTP/1.1
    - GET /search?query=lady+java HTTP/1.1

HTTP POST (1)

- **HTTP POST**
  - Sends data to the server, packaged as part of the message
  - Should be used for programs that change data on the server
- How HTTP POST works
  - Unlike GET, POST includes data in the **message body**, not in the URL.
  - Required headers

    Content-Type: application/x-www-form-urlencoded

    Content-Length: 26 // number of characters
  - Message in the body

    name="xxx"&address="yyy"
  - The structure (paramName1 = "paramValue1"&paramName2=....) is defined by the *x-www-form-urlencoded* standard (Content-Type in the header)

HTTP POST (2)

- **When to use POST?**
  - For operations with side effects
    - Database updates, sending emails etc
  - For file uploads and complex data
    - When multi-part/form data is required
  - For non-ASCII data
    - Handles special characters like accented letters more reliably
  - For large datasets
    - GET URLs can run into length limits (e.g. over 1KB)
  - To hide data from the user
    - Data is not visible in the URL, although the user can still view the page source

HTTP GET vs POST

- What is the difference between these methods?
- When would you use one over another?
  - A safe operation is an operation which does not change the data requested.
  - An idempotent operation is one in which the result will be the same no matter how many times you request it.

  – Use GET for safe and idempotent requests
  – Use POST for neither safe nor idempotent requests

Differences between GET and POST

	GET	POST
Back button	Harmless	Data will be resubmitted
Bookmarked	Can be bookmarked	-
Cached	Can be cached	-
History	Parameters remain in history	-
Data visibility	Visible to everyone in the URL	Not visible
Data length/Data type	Max 2048 characters, only ASCII	No restrictions
Security	Less secure – data is part of the URL	More secure

Other HTTP Methods

- **The HTTP protocol has numerous other methods too:**
  - **HEAD** is just like GET, except it asks the server to return the response headers only
    - useful to check characteristics of a resource without actually downloading it
  - **PUT** for storing data on the server
  - **DELETE** for deleting a resource on the server
  - **OPTIONS** for finding out what the server can do - e.g. switch to secure connections
  - **TRACE** for debugging connections
  - **CONNECT** for establishing a link through a proxy

HTTP Response Status Codes

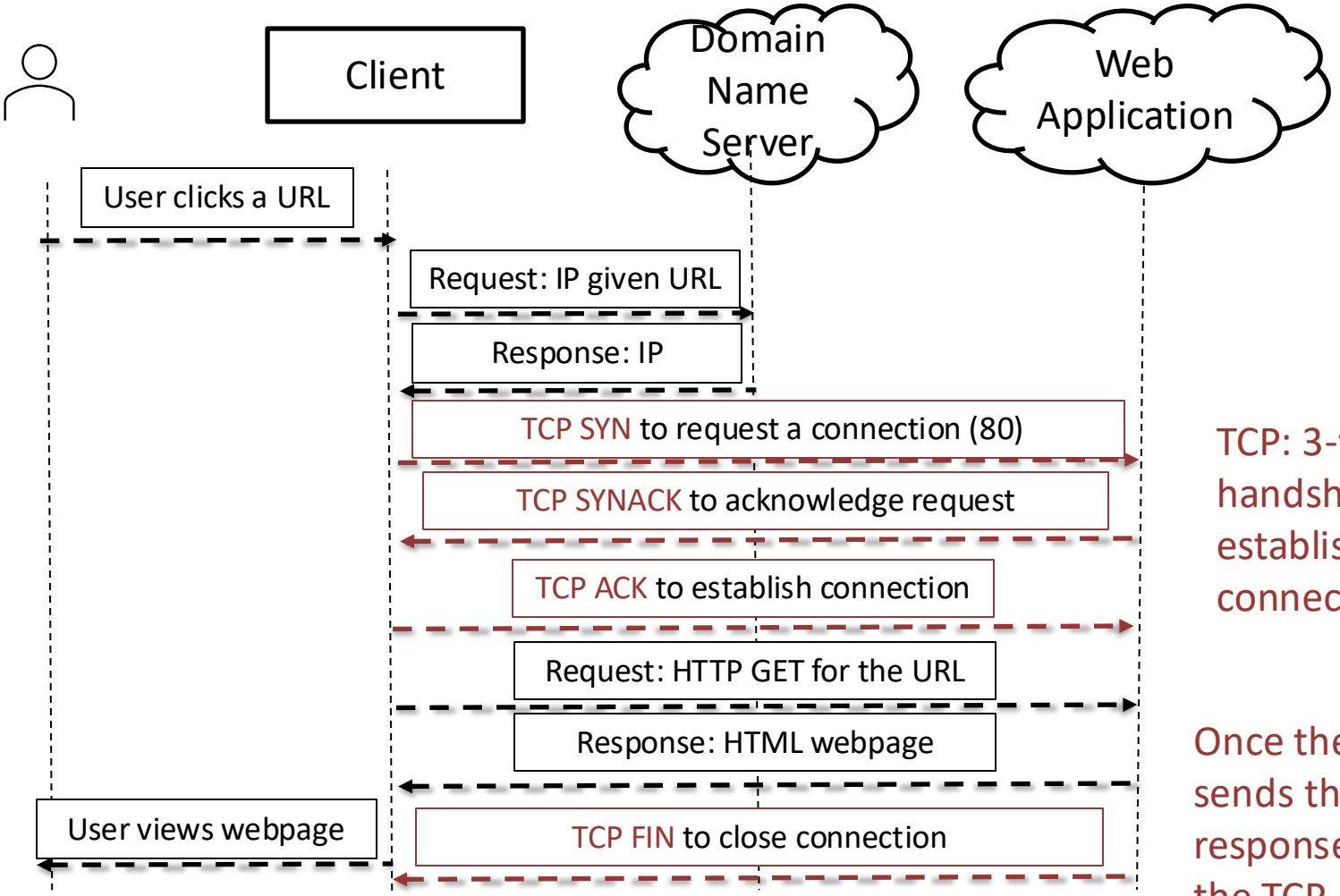
- HTTP responses return status codes in the 1st line of the server-to-client response message

- 200 OK
  – request succeeded, requested object later in this message
- 301 Moved Permanently
  – requested object moved, new location specified later in this message
- 400 Bad Request
  – request message not understood by server
- 404 Not Found
  – requested document not found on this server
- 505 HTTP Version Not Supported

HTTP is stateless

- Under HTTP, communication is as follows:
  - An HTTP client opens a connection and sends a **request message** to an HTTP server
    - Typically the request is either a GET or POST
  - The server then returns a **response message**, usually containing the resource that was requested.
    - Typically, in XHTML, XML, but also in other formats like JSON
  - After delivering the response, the server **closes the connection** making HTTP a stateless protocol
    - i.e. not maintaining any connection information between transactions

HTTP and TCP



TCP: 3-way handshake to establish connection

Once the server sends the HTTP response, it closes the TCP connection

HTTP Statelessness can be a problem

- HTTP does not require the server to retain any information about the client/user
  - All requests are independent

- This is a problem if we want to **maintain a session**
  - Logged-in user
  - Multi-step forms
  - User preferences
  - E-commerce cart

Dealing with Statelessness

Common solutions to overcome statelessness

- Client-side
  - HTTP Cookies
    - See TwD Chapter 10
  - URL encoding to store a session-ID within a URL
- Server-side
  - Server-side Sessions
  - Hidden Form Fields

HTTP Cookies

- How do cookies work?
  - Alice uses browser on device to visit an e-commerce site for the first time
  - When the initial HTTP request arrives at the server, the server creates a **unique ID - the cookie** – and an entry in the backend database
  - Subsequent HTTP requests from Alice to this site will contain the cookie ID, allowing the server to identify Alice
- When to use?
  - Authorization, E-commerce shopping carts
- Privacy concerns
  - Websites can learn a lot about a user
  - Third-party cookies allow the same user to be **tracked across multiple websites**

https://i.redd.it/uh6nfugw5jf51.jpg

Persistent HTTP (1.1)

Persistent HTTP (HTTP1.1):

- The server leaves connection open after sending response
- Subsequent HTTP messages between same client/server sent over open connection
- Time to load a page: As little as one round-trip (request+response) time for all the referenced objects (cutting response time in half)

Further HTTP improvements

- What if the first request takes a really long time?
  - Fetching a large media file
  - All subsequent requests block -> low performance!
- HTTP 2 divides objects to be retrieved into segments
  - A response carrying a large media file is segmented
- HTTP 3 replaces TCP entirely with QUIC (Quick UDP Internet Connections) to improve RTT

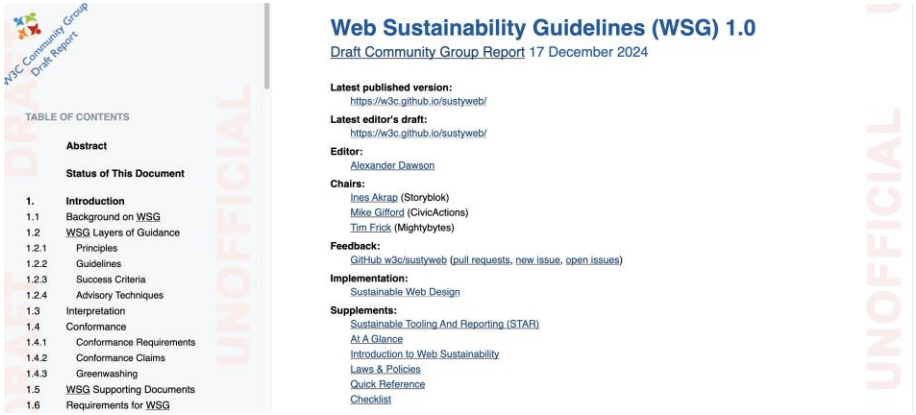
Browser caching

- **Browser caching** stores some data on the browser to avoid frequent transfers from the server
- How does browser caching works?
  - Alice uses browser on device to visit an e-commerce site for the first time.
  - When the first HTTP request arrives at the server, the server responds with the requested resources (HTML, CSS, images, etc.) along with **caching headers**
  - Alice's browser stores these resources in its **local cache**
  - Subsequent HTTP requests from Alice include **conditional GET headers** to check whether the cached resource has changed.
  - If the resource hasn't changed, the server returns a **304 Not Modified** response, allowing the browser to load the resource from the cache without downloading it again.

- When to use?
  - To **improve page load speed**, and reduce server load
- Concerns
  - **Staleness!** If not used properly, caching can result in serving outdated content

Web Sustainability Guidelines

- Web Sustainability Guidelines: a working draft from the W3C



W3C Community Group Draft Report

TABLE OF CONTENTS

Web Sustainability Guidelines (WSG) 1.0

Draft Community Group Report 17 December 2024

Latest published version:
https://w3c.github.io/sustyweb/

Latest editor's draft:
https://w3c.github.io/sustyweb/

Editor:
Alexander Dawson

Chairs:
Ines Akrap (Storyblok)
Mike Gifford (CivicActions)
Tim Frick (Mightybytes)

Feedback:
GitHub w3c/sustyweb (pull requests, new issue, open issues)

Implementation:
Sustainable Web Design

Supplements:
Sustainable Tooling And Reporting (STAR)
At A Glance
Introduction to Web Sustainability
Laws & Policies
Quick Reference
Checklist

https://w3c.github.io/sustyweb/

12 guidelines relevant to Hosting and Infrastructure (Section 4):
https://sustainablewebdesign.org/guideline-categories/hosting-infrastructure-and-systems/

Every HTTP request will result to a response (data transfer) from the server to the client.

Data transfers over the network consume a lot of energy!

Web sustainability and Messaging

- **Guideline #4.2**: Optimize Browser Caching
  - Browser caching reduces the requirement for files to need to be constantly reloaded from the server
  - Significant reduction in data transfers over the network (between the client side and the server side)
  - High impact in energy consumption and making the web greener

Web sustainability and Messaging

- **Guideline #4.3**: Compress your files
  - Remember that, with persistent HTTP, the time to load a page is relevant to the objects being loaded
  - Files/Data/Media need to be transferred to every visitor, incurring significant load on the network
  - Aligns well with the mobile-first principle (see Week 3) - compress media files and offer different resolutions for different devices

Web sustainability and Messaging

- **Guideline #4.4**: Use Error Pages and Redirects Carefully
  - Ensure links are correct so that HTTP requests won't fail
  - Redirect pages only when necessary

Web sustainability and Messaging

- **Guideline #4.7**: Maintain a relevant refresh frequency
  - Only send data from the server when the visitor needs it
  - Rely on client-side caching, or server-side caching as much as possible
  - Prefer manual refreshes

Inter-Process Protocols

Protocols for Inter-Process Communication (IPC)

- HTTP and the other URL schemes are based on the user-agent/client-server model of data
  - Look at Django: it may be a middleware, but it still features a web server (built-in development web server or Nginx/Apache in deployment)
- What about applications in which the code has been distributed about the internet in other ways?
  - e.g. web services, The Cloud
- Solution: Inter Process Communication Protocols
  - IPC techniques are divided into methods for:
    - **message passing**,
    - **synchronization**,
    - **shared memory**, and
    - **remote procedure calls** (RPC)

Legacy methods for IPC

Some legacy techniques:

- **SOAP** – the standard method for Web Services
  - Still used in many legacy systems, necessary to maintain
  - Quite complex to implement
- **XML-RPC** – an XML protocol for describing method calls
  - Simpler and more lightweight than SOAP
  - Can still be found in some older systems

Modern IPC

Modern techniques:

- **RPC:** Remote procedural calls
  - A method running on one machine can call a method running on another
  - gRPC is the most popular framework (created by Google, now open-source) on top of HTTP/2
  - Used at the core of many distributed applications
- **REST:** Representational State Transfer
  - Uses simple HTTP methods to exchange XML or JSON
  - RESTful APIs are implementations of REST – you can build one on your own!
    - Django REST Framework
  - Stateless, easy to integrate, scalable, include features like authentication and caching

gRPC

- gRPC is a modern, open-source RPC framework designed for high-performance communication
- gRPC is built on top of HTTP/2
  - Benefits from multiplexing transmission (load many resources in parallel), compressing headers, and the better connection management
- gRPC supports communication between micro-services written in different languages
  - There is a description of the API contracts (Protocol Buffer)
  - Server and client use stub codes for the methods (Protcol Buffer compiler and gRPC plugins)

gRPC Example

- Client side:

The client uses generated code to call `GetStockQuote("IBM")`

- Server side:

The server implements the `GetStockQuote` method, processes the request and returns the stock price

- Data structures are defined in the `.proto` file, and the Protocol Buffer compiler will generate source code for languages like C++, Python, Go, Java and more

```
syntax = "proto3";

package stock;

// Service definition for obtaining a stock quote.
service StockService {
  // RPC method to get a stock quote for a given symbol.
  rpc GetStockQuote (StockRequest) returns (StockResponse) {}
}

// Request message containing the stock symbol.
message StockRequest {
  string symbol = 1;
}

// Response message containing the stock price.
message StockResponse {
  double price = 1;
}
```

REpresentational State Transfer

- REST is a rather different way of using Web Services
- It avoids specific protocols, rather expecting:
  - the service to send a file/resource, and,
  - the client to pick it apart (using SAX, DOM, XSLT, etc.) – *more in Lecture 6/Client-side scripting*
- REST builds directly on HTTP
  - sending simple GET or POST requests and expecting the result to be XML/JSON/ETC rather than XHTML

Why REST

- It is simpler than SOAP or XML-RPC,
- In comparison to SOAP and XML-RPC, it is designed to transport resources
- Requires some additional coding to handle either end
  - i.e. XML Parsing
- In a sense, HTTP itself is a **ReSTful** application, with the HTTP being interpreted by the server and user agent

REST Example

- Assume that the resource is represented by http://www.example.org/stock/IBM
- The server should return data in XML format

```
import xml.etree.ElementTree as ET
from urllib.request import urlopen

# Define the RESTful API endpoint for the stock quote
url = "http://www.example.org/stock/IBM"

# urlopen will generate a GET request
with urlopen(url) as response:
    xml_data = response.read()

# Parse the XML data
root = ET.fromstring(xml_data)
# Assuming the XML structure contains <price> within <stock>
price = root.find('price').text
```

Request Message

Response Message

ReST Examples in Python

Search Spotify using GET

```
from urllib.request import urlopen
from urllib.error import URLError

url = ‘https://api.spotify.com/v1/search?type=artist&q=snoop’

try:
   response = urlopen( request )
   print(response.read( 1000 ))

except URLError as err:
   print(err.reason)
```

No params required, generates a GET request

WARNING: A proxy will need to be used if you try this in the university and you need an OAUTH2 key to access it

Add to a track to a playlist using POST

```
import json
from urllib.request import Request, urlopen

# Define the API endpoint for creating a playlist
url =“http://api.musicfun.com/v1/playlists”

playlist_data = {
    "name": "My Awesome Playlist",
    "description": "A collection of my favorite tunes.",
    "tracks": ["track1", "track2", "track3"]
}

# Convert the playlist data to a JSON string and encode it to bytes
encoded_data = json.dumps(playlist_data).encode('utf-8')

# Create a Request object with the JSON data (this makes it a POST request)
req = Request(url, data=encoded_data)
req.add_header("Content-Type", "application/json")

# Send the POST request and read the response
with urlopen(req) as response:
    result = response.read().decode('utf-8')

print("Response from MusicFun API:")
print(result)
```

The Request object now has some encoded data – this is a POST request

Summary

- Networking basics
  - OSI vs TCP/IP models – layered architecture for web communication
  - Client-Server model – clients request resources from servers (web servers, middleware)
- HTTP fundamentals
  - Request/Response pattern
  - GET to retrieve data, POST to send data
  - Stateless protocol: HTTP transactions are independent
- Inter-Process Communication
  - REST: Lightweight, resource-oriented, built on standard HTTP methods
- Extras:
  - Performance/Sustainability: persistent connections, modern HTTP, caching
  - Real-world applications: RESTful APIs and gRPC

Half-way there…

- Next week (Week 5), you will be about half-way through the course
  - One quiz done, two to go
  - First part of group coursework done, second part to go
  - Keep up the good work!
- Use communication channels! Come to the labs! Ask questions!
  - Padlet is your friend!
- See you on Week 6!